

PROGRAMLAMA DİLLERİNİN KAVRAMLARI

Concepts of Programming Languages — Sebesta, 12. Baskı

BÖLÜM 1 Preliminaries (Ön Bilgiler)

Midterm Çalışma Rehberi

Tüm kavramlar • Örnekler • Karşılaştırma tabloları • Quiz & Çözümleri

Kaynak: Robert W. Sebesta — Concepts of Programming Languages, 12. Baskı (2019)

İçindekiler

1.1 Programlama Dili Kavramlarını Öğrenmenin Nedenleri

1.2 Programlama Alanları

1.3 Dil Değerlendirme Kriterleri

1.3.1 Readability (Okunabilirlik)

1.3.2 Writability (Yazılabilirlik)

1.3.3 Reliability (Güvenilirlik)

1.3.4 Cost (Maliyet)

1.4 Dil Tasarımına Etkiler

1.4.1 Von Neumann Mimarisi

1.4.2 Programlama Tasarım Metodolojileri

1.5 Dil Kategorileri

1.6 Dil Tasarımı Ödünleştirmeleri

1.7 Uygulama Yöntemleri

1.7.1 Compilation (Derleme)

1.7.2 Pure Interpretation (Saf Yorumlama)

1.7.3 Hybrid Implementation Systems

1.7.4 Preprocessors (Ön-işleyiciler)

1.8 Programlama Ortamları

— Çıkması Muhtemel Sınav Soruları

— Ünite Sonu Soruları ve Tam Çözümleri

1.1 Reasons for Studying Concepts of Programming Languages

(Programlama Dili Kavramlarını Öğrenmenin Nedenleri)

Programlama dili kavramlarını incelemek, bir yazılım geliştiriciye yalnızca teknik beceri katmaz; aynı zamanda düşünme ve problem-çözme kapasitesini de genişletir. Kitap, bu çalışmanın neden değerli olduğunu altı temel gerekçeyle açıklamaktadır.

i 1. Increased capacity to express ideas (Fikir ifade kapasitesinin artması)

Bir programcının düşünce derinliği, kullandığı dilin ifade gücüyle doğru orantılıdır. Başka dillerin yapılarını bilen bir programcı, kendi dilinde bu yapıları simüle ederek kısıtlamaları aşabilir. Örneğin, Perl'deki associative array (ilişkisel dizi) yapısını öğrenen bir C programcısı benzer veri yapılarını C'de de inşa edebilir.

i 2. Improved background for choosing appropriate languages (Uygun dil seçimi için gelişmiş altyapı)

Pek çok programcı alışkanlıkla değil bilinçli seçimle dil belirleyemez. Geniş bir dil kavramları repertuarına sahip olan kişi, projeye en uygun dili seçebilir. Bir dilin özelliğini simüle etmek mümkün olsa da, tasarımına entegre edilmiş özelliği kullanmak daha zarif, verimli ve güvenlidir.

✓ 3. Increased ability to learn new languages (Yeni dil öğrenme kapasitesinin artması)

Programlama henüz genç bir disiplindir; diller sürekli evrilir. Temel kavramları kavramış biri yeni bir dili çok daha hızlı öğrenir. Tıpkı anadilini iyi bilmenin ikinci dil öğrenimini kolaylaştırması gibi. TIOBE endeksi, popüler dillerin sürekli değiştiğini göstermektedir — bu da kesintisiz öğrenmeyi zorunlu kılar.

i 4. Better understanding of the significance of implementation (Uygulama detaylarının önemi)

Bir dilin neden belirli biçimde tasarlandığını anlamak, onu daha etkin kullanmayı sağlar. Örneğin, subprogram call (alt program çağırısı) maliyetini bilen bir programcı, küçük ama sıkça çağrılan alt programların performans cezasını öngörebilir. Bu bilgi aynı zamanda compiler design (derleyici tasarımı) çalışmaları için de temel oluşturur.

✓ 5. Better use of languages that are already known (Bilinen dillerin daha verimli kullanımı)

Büyük ve karmaşık dillerin tüm özellikleri genellikle tam olarak bilinmez. Kavramsal eğitim, programcıların daha önce fark etmedikleri dil özelliklerini keşfetmesine ve kullanmasına yardımcı olur.

⚠ 6. Overall advancement of computing (Bilişimin genel ilerlemesine katkı)

Dil kavramlarına hâkim geliştiriciler, topluluk genelinde daha bilinçli dil tercihlerine yol açarlar. Tarihi örnek: ALGOL 60, Fortran'dan çok daha zarif ve yapısal olmasına karşın, tasarımcıların kavramsal derinlikten yoksun olması nedeniyle yaygınlaşamamıştır.

Önemli Çıkarımlar (Key Takeaways)

- ✓ Dil kavramları bilgisi, salt teknik değil entelektüel bir yatırımdır.
- ✓ Yeni dil öğrenimini hızlandırır; temel kavramlar transfer edilebilirdir.
- ✓ Dil seçimini alışkanlığa değil bilinçli değerlendirmeye dayandırmayı sağlar.
- ✓ Uygulama detaylarını anlamak, performans odaklı ve hata-az yazılım geliştirmeye katkı sağlar.
- ✓ Bakım ve hata ayıklama süreçlerinde implementasyon bilgisi kritik bir avantaj sunar.

1.2 Programming Domains (Programlama Alanları)

Bilgisayarlar son derece geniş bir uygulama yelpazesinde kullanılmaktadır — nükleer santral kontrolünden mobil oyunlara kadar. Bu çeşitlilik, birbirinden farklı hedeflere sahip programlama dillerinin ortaya çıkmasına yol açmıştır. Her uygulama alanı kendine özgü veri yapısı, kontrol yapısı ve etkinlik gereksinimlerine sahiptir.

Tablo 1.1 — Programlama Alanları Özeti

Alan	Dönem	Temel Gereksinim	Öncü Dil(ler)
Scientific Applications (Bilimsel)	1940'lar-	Kayan nokta aritmetiği, dizi/matris işlemleri	Fortran, ALGOL 60
Business Applications (İş)	1950'ler-	Ondalık aritmetik, raporlama, karakter verisi	COBOL
Artificial Intelligence (Yapay Zeka)	1950'ler-	Sembolik hesaplama, bağlı liste yapıları	Lisp, Prolog, Python
Web Software (Web Yazılımı)	1990'lar-	Dinamik içerik, çoklu dil entegrasyonu	JavaScript, PHP, Java, HTML

1.2.1 Scientific Applications (Bilimsel Uygulamalar)

İlk dijital bilgisayarlar (1940'ların sonu - 1950'lerin başı) bilimsel amaçlı tasarlanmıştır. Bu uygulamalar; **simple data structures** (*basit veri yapıları*) (diziler, matrisler) kullanırken yoğun **floating-point arithmetic** (*kayan nokta aritmetiği*) hesaplamaları gerektirirdi. Kontrol yapısı genellikle sayma döngüsü ve seçim ifadelerinden oluşurdu. Assembly diliyle doğrudan rekabet ettiklerinden, **efficiency** (*verimlilik*) birincil tasarım ölçütüydü. Bu alan için geliştirilen ilk yüksek seviyeli dil **Fortran**'dır.

1.2.2 Business Applications (İş Uygulamaları)

1950'lerde iş uygulamaları için özel donanım ve diller geliştirildi. İlk başarılı iş dili **COBOL** (*Common Business-Oriented Language*)'dır (1960). İş dilleri üç temel özellikte karakterize edilir:

- Ayrıntılı rapor üretimi (**elaborate report generation** (*kapsamlı rapor üretimi*))
- **Decimal numbers** (*Ondalık sayı*) ve karakter verisi için hassas tanımlama ve depolama
- **Decimal arithmetic operations** (*Ondalık aritmetik işlemler*) belirtebilme kapasitesi

1.2.3 Artificial Intelligence (Yapay Zeka)

YZ uygulamaları **symbolic computation** (*sembolik hesaplama*) ile karakterize edilir: sayılar yerine *isimler* ve *semboller* manipüle edilir. Veri yapısı olarak **linked list** (*bağlı liste*) dizilere göre daha uygun ve yaygındır. Bu alan için geliştirilen ilk yaygın dil **Lisp** (1959) işlevsel dildir. 1970'lerin başında **logic programming** (*mantık programlama*) paradigmasıyla **Prolog** ortaya çıkmıştır. Günümüzde **Python** da YZ uygulamalarında yaygın olarak kullanılmaktadır.

1.2.4 Web Software (Web Yazılımı)

Web, farklı kategorilerdeki dillerin bir koleksiyonunu barındırır. İçerik düzeni için **markup language** (*işaretleme dili*) olan HTML (programlama dili değildir); dinamik içerik için **scripting language** (*betik dili*) JavaScript ve PHP; genel amaçlı programlama için Java kullanılır. HTML'e programlama kapasitesi katan uzantılar da mevcuttur: JSTL ve XSLT bunların başında gelir.

Önemli Çıkarımlar (Key Takeaways)

- ✓ Her programlama alanının kendine özgü veri yapısı ve kontrol ihtiyaçları vardır.
- ✓ Fortran → Bilimsel; COBOL → İş; Lisp/Prolog → YZ; JavaScript/PHP → Web
- ✓ Dil seçiminde uygulama alanını göz önünde bulundurmak verimlilik açısından kritiktir.
- ✓ YZ'de symbolic computation (sembolik hesaplama) sayısal hesaplamanın önüne geçer.

1.3 Language Evaluation Criteria (Dil Değerlendirme Kriterleri)

Programlama dillerini değerlendirebilmek için nesnel ölçütler gerekmektedir. Kitap dört temel kriteri öne çıkarmaktadır: **Readability**, **Writability**, **Reliability** ve **Cost**. Bu kriterler hem dil tasarımcılarına hem de dil kullanıcılarına rehberlik eder; ancak birbiriyle çelişebildiklerini başından not almak önemlidir.

Tablo 1.2 — Değerlendirme Kriterleri ve Etkileyen Özellikler (Tablo 1.1 — Sebesta)

Özellik (Characteristic)	Readability (Okunabilirlik)	Writability (Yazılabilirlik)	Reliability (Güvenilirlik)
Simplicity (Basitlik)	●	●	●
Orthogonality (Ortogonallık)	●	●	●
Data types (Veri tipleri)	●	●	●
Syntax design (Sözdizim tasarımı)	●	●	●
Support for abstraction (Soyutlama desteği)	—	●	●
Expressivity (İfade gücü)	—	●	●
Type checking (Tür denetimi)	—	—	●
Exception handling (İstisna yönetimi)	—	—	●
Restricted aliasing (Kısıtlı takma ad)	—	—	●

1.3.1 Readability (Okunabilirlik)

Okunabilirlik; bir programın ne kadar kolay okunup anlaşılabilirliğinin ölçüsüdür. 1970 öncesinde yazılım geliştirme ağırlıklı olarak kod *yazmak* olarak algılanıyordu. 1970'lerde **software life-cycle** (*yazılım yaşam döngüsü*) kavramının gelişmesiyle bakım maliyetleri gündeme geldi ve **okunabilirlik bir kalite ölçütü** haline geldi. Bu dönüşüm, dil tasarımını **makine odaklılıktan insan odaklılığa** taşıdı.

1.3.1.1 Overall Simplicity (Genel Basitlik)

Çok sayıda temel yapıya sahip bir dil öğrenmek güçtür. Programcılar büyük dillerde genellikle bir alt küme öğrenir. Okunabilirlik sorunu tam da bu noktada ortaya çıkar: yazar ile okuyucunun öğrendiği alt kümeler farklılaşabilir.

i Feature Multiplicity (Özellik Çokluluğu)

Aynı işlemi birden fazla yolla yapabilmek okunabilirliği düşürebilir. Örneğin Java'da bir tam sayı değişkenini artırmanın **dört yolu** vardır:

Java'da integer artırma — dört eşdeğer (bazı bağlamlarda farklı) yol:

```
count = count + 1 // Açık atama
count += 1 // Bileşik atama (compound assignment)
count++ // Sonek artırma (postfix increment)
++count // Önek artırma (prefix increment)
```

Operator overloading (*Operatör aşırı yükleme*): Tek bir operatör sembolünün birden fazla anlamı olması kendi başına sorun değildir (+ hem tamsayı hem ondalıklı toplama için kullanılabilir). Ancak kullanıcı tanımlı anlamsız aşırı yüklemeler okunabilirliği ciddi biçimde zedeler.

⚠ **Aşırı Basitlik de Zararlıdır**

Assembly dili son derece basittir; ancak karmaşık kontrol yapılarından yoksun olduğu için okunabilirlik çok düşüktür. Benzer şekilde, yetersiz kontrol ve veri yapısı desteği sunan yüksek seviyeli diller de aynı sorunu yaşar.

1.3.1.2 Orthogonality (Ortogonalite)

Orthogonality (*Ortogonalite*); az sayıda temel yapının az sayıda kuralla birleştirilerek dilin tüm kontrol ve veri yapılarını oluşturabilmesi anlamına gelir. Kritik özellik: her olası kombinasyonun **yasal ve anlamlı** olması. Terim, matematikteki dik (orthogonal) vektörler kavramından — birbirinden bağımsız olmak — türetilmiştir.

Tablo 1.3 — Orthogonality Örneği: IBM vs VAX Toplama Komutları

Mimari	Komut(lar)	Operandlar	Orthogonal mı?	Sonuç
IBM Mainframe	A Reg, Mem AR Reg, Reg	2/4 kombinasyon geçerli	✗ Hayır	Kısıtlı, daha zor öğrenilir
VAX	ADDL op1, op2	4/4 kombinasyon geçerli (register veya memory)	✓ Evet	Esnek, daha az kural

✓ **Orthogonality ve Simplicity Birlikte Çalışır**

Ortogonal tasarım → daha az kural istisnası → daha yüksek düzenlilik → daha kolay öğrenme ve daha iyi okunabilirlik.

⚠ **Aşırı Orthogonality Sorun Yaratır**

ALGOL 68 tarihte en ortogonal dil olarak kabul edilir: her yapı bir türe sahiptir, kısıtlama yoktur. Bu özgürlük saçma kombinasyonlara ve gereksiz karmaşıklığa yol açar. Doğru denge: 'az sayıda primitive + sınırlı orthogonality'.

i **C'de Orthogonality Eksikliği — Örnekler**

- Fonksiyonlar struct döndürebilir; **ama array döndüremez.**
- Parametreler değer ile geçilir; **ama array'lar referans ile geçilir.**
- static** kelimesi bağlama göre farklı anlam taşır (compile-time değişken vs. dosya kapsamı).

1.3.1.3 Data Types (Veri Tipleri)

Yeterli **data type** (*veri tipi*) desteği okunabilirliği artırır. Klasik örnek: C'nin ilk sürümünde Boolean tipi yoktu. Programcı `timeout = 1` yazmak zorundaydı; oysa Boolean destekle `timeout = true` anlamı açık ve nettir.

1.3.1.4 Syntax Design (Sözdizim Tasarımı)

Syntax (*Sözdizim*) tasarımının okunabilirliğe etkisi büyüktür. İki temel konu öne çıkmaktadır:

- Special words (Özel kelimeler):** Ada'nın *end if*, *end loop* gibi ayırt edici kapatıcı sözcükleri hangi bloğun kapandığını netleştirir. C'nin tek tip '}' karakteri ise belirsizlik yaratabilir. Bu, rezerve kelime sayısı (Java: az → basit) ile okunabilirlik (Ada: çok → net) arasındaki klasik çelişkiyi örnekler.
- Form and meaning (Biçim ve anlam):** Sözdiziminin semantiği (anlamı) yansıtması gerekir. C'de *static* kelimesi bağlama göre farklı anlam taşır — bu ortogonallik ihlalidir ve okunabilirliği zedeler.

1.3.2 Writability (Yazılabilirlik)

Writability (*Yazılabilirlik*); seçilen problem alanı için program yazmanın ne kadar kolay olduğunun ölçüsüdür. Okunabilirlik kriterleriyle büyük örtüşme gösterir: program yazarken daha önce yazılan kısımlar sık sık yeniden okunur. Her iki kriter de problem alanı bağlamında değerlendirilmelidir.

i Bağlam Örneği: VB vs. C

GUI (Grafical User Interface / Grafik Kullanıcı Arayüzü) oluşturmak için Visual BASIC çok daha yüksek writability sunar. Sistem programlaması (işletim sistemi vb.) için C üstündür. İki kriteri 'haksız' şartlarda kıyaslamak anlamsızdır.

1.3.2.1 Simplicity and Orthogonality

Az sayıda primitive yapı + tutarlı kombinasyon kuralları = yüksek writability. Büyük ve karmaşık bir dilde bazı özellikler yanlış kullanılır veya hiç kullanılmaz. Hoare (1973): bilinmeyen özellikler yanlışlıkla kullanılabilir ve tuhaf sonuçlar doğurabilir.

1.3.2.2 Expressivity (İfade Gücü)

Expressivity (*İfade gücü*): dilin hesaplamaları ne kadar kısa ve uygun biçimde ifade edebildiğinin ölçüsüdür. Örnekler:

- C'de `count++` ← `count = count + 1` yerine daha kısa
- Ada'nın **and** **then** operatörü → **short-circuit evaluation** (*kısa-devre değerlendirme*) için zarif bir yol
- Java'nın **for** deyimi → while ile de yapılabilir, ama for daha az hata eğilimli

i APL ve Aşırı Expressivity

APL dili, dizi işlemleri için son derece güçlü operatörler sunar. Bu, writability'yi doruk noktasına taşır — ama aynı zamanda readability'yi yerle bir eder. McCracken (1970): dört satırlık bir APL programını anlamak için dört saat harcadım.

1.3.3 Reliability (Güvenilirlik)

Bir program, **tüm koşullarda** spesifikasyonuna uygun davrandığında güvenilir sayılır. Dört alt faktör güvenilirliği etkiler:

1.3.3.1 Type Checking (Tür Denetimi)

Type checking (*Tür denetimi*); bir programdaki tip hatalarının derleme veya çalışma zamanında tespit edilmesidir. **Compile-time type checking** (*Derleme zamanı tür denetimi*) tercih edilir çünkü:

- Hataların erken tespiti → düzeltme maliyeti düşer
- Run-time type checking performans cezası getirir

× Tarihi Hata: Orijinal C'de Tür Denetimi Yoktu

C'nin orijinal sürümünde subprogram parametrelerinin tipleri kontrol edilmiyordu. Bir `int` değişken, `float` bekleyen bir fonksiyona gönderildiğinde hata verilmezdi. 23'ün tam sayı bit dizisi ile kayan nokta bit dizisi birbirinden tamamen farklıdır; bu durum tahmin edilemez sonuçlar üretir ve tanınması son derece güçtür.

1.3.3.2 Exception Handling (İstisna Yönetimi)

Exception handling (*İstisna yönetimi*): programın çalışma zamanı hatalarını yakalayıp düzeltici önlem alarak çalışmaya devam edebilmesidir. Ada, C++, Java, C# kapsamlı istisna yönetimi sunar. C'de bu destek neredeyse hiç yoktur — bu C programlarının güvenilirliğini olumsuz etkiler.

1.3.3.3 Aliasing (Takma Adlama)

Aliasing (*Takma adlama*): aynı bellek hücresine iki veya daha fazla farklı isimle erişebilmektir. Tehlikeli kabul edilir: birinin değerini değiştirmek otomatik olarak diğerini de değiştirir. Çoğu dilde pointer veya referans aracılığıyla aliasing mümkündür. Bazı diller aliasing'i kısıtlar; bazıları veri soyutlama eksikliklerini gidermek için kasıtlı kullanır.

1.3.3.4 Readability and Writability (Okunabilirlik ve Yazılabilirlik)

Her iki kriter de güvenilirliği doğrudan etkiler. Algoritmaları doğal biçimde ifade edemeyen bir dilde programcı zorunlu olarak yapay yaklaşımlar kullanır; bu da hata olasılığını artırır. Yazması zor program → hata oranı yüksek. Okuması zor program → bakımı güç ve hatalı modifikasyon riski yüksek.

1.3.4 Cost (Maliyet)

Bir programlama dilinin toplam maliyeti birden fazla bileşenden oluşur:

Tablo 1.4 — Dil Maliyeti Bileşenleri

Maliyet Bileşeni	Açıklama	Belirleyen Faktör
Training cost (Eğitim maliyeti)	Programcıları dili kullanmaya hazırlamak	Simplicity, Orthogonality
Writing cost (Yazım maliyeti)	Programları dilde geliştirmek	Writability
Execution cost (Yürütme maliyeti)	Programların çalışma hızı	Run-time type check sayısı
Compilation cost (Derleme maliyeti)	Kaynak kodun derlenmesi	Optimization seviyesi
Poor reliability cost (Düşük güvenilirlik)	Hata kaynaklı zararlar	Reliability özellikleri
Maintenance cost (Bakım maliyeti)	Düzeltilme ve yeni özellik ekleme; geliştirme maliyetinin 2-4 katı!	Readability (birincil)

⚠ **Kritik Not: Bakım Maliyeti**

Büyük ve uzun ömürlü yazılım sistemlerinde bakım maliyeti, geliştirme maliyetinin **2 ila 4 katına** ulaşabilir (Sommerville, 2010). Bu nedenle readability ve writability, maliyet açısından en belirleyici değerlendirme kriterleridir.

Ek kriterler: **Portability** (*Taşınabilirlik*) — standardizasyon derecesiyle belirlenir; **Generality** (*Genellik*) — geniş uygulama yelpazesine uygulanabilirlik; **Well-definedness** (*İyi tanımlanmışlık*) — resmi tanımlama belgesinin eksiksizliği ve hassasiyeti.

i Perspektif Farkı

- **Dil uygulayıcıları:** yapıların gerçekleştirme güçlüğüne odaklanır.
- **Dil kullanıcıları:** önce writability, sonra readability.
- **Dil tasarımcıları:** zarafet ve yaygın benimseme kapasitesi.

Önemli Çıkarımlar (Key Takeaways)

- ✓ Dört temel kriter: Readability · Writability · Reliability · Cost
- ✓ Orthogonality: az kural, her kombinasyon yasal ve anlamlı — okunabilirlik ve yazılabilirlik için kritik.
- ✓ Type checking erken yapılırsa (compile-time) maliyeti düşürür, güvenilirliği artırır.
- ✓ Aliasing tehlikeli bir özelliktir; güvenilirliği olumsuz etkiler.
- ✓ Bakım maliyeti geliştirme maliyetinin 2-4 katı olabilir → readability hayati önem taşır.
- ✓ Criteria birbiriyle çelişebilir: Java (güvenilirlik > hız), APL (yazılabilirlik > okunabilirlik).

1.4 Influences on Language Design (Dil Tasarımına Etkiler)

Dil tasarımını etkileyen iki temel faktör bulunmaktadır: **computer architecture** (bilgisayar mimarisi) ve **programming design methodologies** (programlama tasarım metodolojileri). Bu iki etken, geçmiş altmış yılın dil evrimini şekillendiren başlıca güçlerdir.

1.4.1 Computer Architecture (Bilgisayar Mimarisi)

Son 60 yılın popüler dillerinin büyük çoğunluğu, **von Neumann architecture** (*von Neumann mimarisi*) etrafında tasarlanmıştır. Bu mimariyi kullanan dillere **imperative languages** (*buyurumsal diller*) denir.

i Von Neumann Mimarisinin Temel Özellikleri

- Veri ve programlar **aynı bellekte** saklanır.
- **CPU** (*Merkezi İşlem Birimi*) bellekten ayrıdır.
- Komutlar ve veriler **piping** ile bellekten CPU'ya, sonuçlar geri belleğe taşınır.
- 1940'lardan bu yana neredeyse tüm dijital bilgisayarlar bu mimariyi kullanır.

Bu mimari, imperative dillerin merkezi özelliklerini doğrudan biçimlendirir:

- **Variables** (*Değişkenler*): bellek hücrelerini modeller
- **Assignment statements** (*Atama ifadeleri*): piping operasyonunu temsil eder
- **Iterative repetition** (*Yinelemeli tekrar*): döngüler bu mimaride en verimli tekrar biçimidir

Fetch-Execute Cycle (Getir-Yürüt Döngüsü)

Von Neumann bilgisayarda program yürütmesi **fetch-execute cycle** (*getir-yürüt döngüsü*) ile gerçekleşir. Bir sonraki yürütülecek komutun adresi **program counter** (*program sayacı*) adı verilen bir yazmaçta tutulur.

Fetch-Execute Cycle — Algoritma:

```
program_counter'i başlat
sonsuz döngü:
  program_counter'in gösterdiği komutu bellekten AL (fetch)
  program_counter'i bir sonraki komuta ARTIR (increment)
  komutu ÇÖZÜMLE (decode) – ne yapılacağı belirlenir
  komutu YÜRÜT (execute)
döngü sonu
```

i Von Neumann Bottleneck (Darboğazı)

Bellek ile CPU arasındaki bağlantı hızı, genellikle tüm sistem performansının belirleyicisidir. Komutlar CPU'ya taşınmadan yürütülemez. Bu darboğaz, **parallel computer (paralel bilgisayar)** araştırmalarının başlıca motivasyonu olmuştur.

Functional Languages (Fonksiyonel Diller) ve Von Neumann

Fonksiyonel (ya da **applicative** (*uygulayıcı*)) diller, hesaplamayı fonksiyon uygulamalarıyla yapar. Değişken, atama ve iterasyon gerektirmezler. Birçok bilim insanı fonksiyonel dillerin üstünlüklerini savunmuştur; ancak non-von Neumann bir donanım olmadan imperative dillerin yerini almanın güçtür. John Backus (1978) — Fortran'ın baş tasarımcısı — bu gerçeği en güçlü şekilde ifade edenlerden biridir.

1.4.2 Programming Design Methodologies (Tasarım Metodolojileri)

1960'ların sonu – 1970'lerin başında, yazılım geliştirme süreçleri yoğun biçimde yeniden ele alındı. Temel neden: donanım maliyetleri düşerken programcı maliyetleri artıyordu ve çözülen problemler giderek büyüyordu (uydu yörünge hesabından global havayolu rezervasyon sistemlerine).

Tablo 1.5 — Programlama Tasarım Metodolojilerinin Evrimi

Dönem	Metodoloji	Temel Odak	Dil Örneği
1970'ler	Top-down design Stepwise refinement	Kontrol yapıları; goto'dan kaçınma	Pascal, ALGOL
1970'lerin sonu	Data-oriented design (Veri odaklı)	Abstract Data Types (Soyut Veri Tipleri)	SIMULA 67, Ada
1980'lerin başı+	Object-Oriented Design (Nesne Yönelimli)	Veri soyutlama + kalıtım + dinamik bağlama	Smalltalk, Java, C++
Günümüz	Concurrency (Eş) Zamanlılık	Paralel program birimleri	Java, C#, Go

Object-oriented metodoloji üç temel unsurdan oluşur: (1) **data abstraction** (*veri soyutlama*) — veriye erişimi kapsüller; (2) **inheritance** (*kalıtım*) — mevcut yazılımı yeniden kullanmayı sağlar; (3) **dynamic method binding** (*dinamik metod bağlama*) — kalıtımı daha esnek kılar. Smalltalk bu üç özelliği destekleyen ilk dildir.

Önemli Çıkarımlar (Key Takeaways)

- ✓ Von Neumann mimarisi → imperative dillerin temel yapısını (değişken, atama, döngü) belirledi.
- ✓ Von Neumann bottleneck: bellek-CPU bant genişliği tüm sistemin hız sınırındır.
- ✓ Fetch-execute cycle: program counter → komut alımı → çözümleme → yürütme.
- ✓ Tasarım metodolojileri sırası: yapısal → veri odaklı (ADT) → nesne yönelimli → eş zamanlı.
- ✓ OOP'nin üç temel özelliği: data abstraction + inheritance + dynamic method binding.
- ✓ Smalltalk bu üç özelliği destekleyen ilk dildir.

1.5 Language Categories (Dil Kategorileri)

Programlama dilleri genellikle dört temel kategoride incelenir. Kitap bu sınıflamayı belirli nüanslarla sunar:

Kategori	Açıklama	Örnekler	Not
Imperative Languages (Buyurumsal Diller)	Von Neumann mimarisine dayalı; değişken, atama ve döngü merkezlidir.	<i>C, C++, Java, Python, C#</i>	✓ OOP destekleyenler bu kategoriden türer; ayrı kategori sayılmaz.
Functional Languages (Fonksiyonel Diller)	Hesaplama fonksiyon uygulamalarıyla yapılır. Değişken ve atama yoktur.	<i>Lisp, Scheme, ML, Haskell, F#</i>	Verimlilik kaygısı yaygınlaşmalarını önlemektedir.
Logic Languages (Mantık Dilleri)	Kural tabanlıdır (rule-based). Programcı algoritma sıralaması belirtmez; sistem karar verir.	<i>Prolog</i>	Radikalce farklı bir geliştirme yaklaşımı gerektirir.
Markup/Programming Hybrid Languages (Hibrid Diller)	HTML/XML'e programlama kapasitesi eklenmiş dillerdir.	<i>JSTL, XSLT</i>	Tam programlama dilleriyle kıyaslanamaz; Bölüm 2'den sonra ele alınmaz.

i Scripting Languages (Betik Dilleri) Ayrı Bir Kategori Değildir

Perl, JavaScript, Ruby gibi diller zaman zaman scripting dil olarak anılır. Ancak bunları birleştiren şey dil tasarımı değil, uygulama yöntemidir (kısmi veya tam yorumlama). Bu diller temelde **imperative** dillerdir.

Önemli Çıkarımlar (Key Takeaways)

- ✓ Dört temel kategori: Imperative · Functional · Logic · Markup/Hybrid
- ✓ OOP desteği olan diller imperative kategoriden türer; ayrı kategori sayılmaz.
- ✓ Logic dillerde programcı algoritma sıralaması belirtmez; sistem karar verir.
- ✓ Scripting dilleri temelde imperative'dir; ortak paydaları uygulama yöntemidir.

1.6 Language Design Trade-Offs (Dil Tasarımı Ödünleşimleri)

Dil değerlendirme kriterleri birbiriyle çelişebilir. Hoare (1973): 'Uzlaştırılması ve karşılanması gereken pek çok önemli ama çelişen kriter bulunmaktadır; bu nedenle dil tasarımı büyük bir mühendislik görevidir.'

Tablo 1.6 — Çelişen Kriterler ve Tasarım Kararları

Çelişen Kriterler	Ödünleşim	Örnek Dil	Tercih
Reliability vs. Execution Cost	Güvenilirlik için hız ödenir	Java vs C	Java: dizi index kontrolü → güvenilir ama yavaş C: kontrol yok → hızlı ama riskli
Writability vs. Readability	İfade gücü okunabilirliği azaltır	APL	APL: çok güçlü operatörler → yazması kolay, okuması neredeyse imkânsız
Writability vs. Reliability	Esneklik güvenilirliği azaltır	C++ vs Java	C++ pointer esnekliği → writability ↑, reliability ↓ Java pointer yok → reliability ↑

Önemli Çıkarımlar (Key Takeaways)

- ✓ Dil tasarımı mühendislik ödünleşimlerini zorunlu kılar — mükemmel dil yoktur.
- ✓ Java: güvenilirlik > performans; C: performans > güvenilirlik.
- ✓ APL: writability > readability — 4 satırlık program 4 saatte anlaşılıyor.
- ✓ Çelişen kriterler bazen ince, bazen apaçık ortadadır.

1.7 Implementation Methods (Uygulama / Gerçekleme Yöntemleri)

Programlama dilleri üç genel yöntemle gerçekleştirilebilir. Her uygulama sistemi, işletim sistemi üzerine katmanlanmış bir **virtual computer** (*sanal bilgisayar*) oluşturur.

Tablo 1.7 — Gerçekleme Yöntemlerinin Karşılaştırması

Yöntem	Çalışma Prensipleri	Hız	Avantaj	Dezavantaj	Örnekler
Compilation (Derleme)	Kaynak → makine dili; bir kez çeviri	En hızlı	Program bir kez derlenir, defalarca hızla çalışır	Derleme süreci ek adım	C, C++, COBOL
Pure Interpretation (Saf Yorumlama)	Yorumlayıcı kaynak kodu doğrudan yürütür	10-100x yavaş	Kaynak düzeyinde hata mesajları; kolay debug	Yavaş; her çalıştırmada komut yeniden çözülür	JavaScript, PHP (eski)
Hybrid Implementation (Hibrid)	Kaynak → ara dil → yorumlama	Orta hız	Kaynak bir kez çözülür; açık platforma taşınır	Makine kodundan yavaş	Java (byte code), Perl, .NET (JIT)

1.7.1 Compilation (Derleme)

Derleme yönteminde kaynak program **source language** (*kaynak dil*)'den makine diline çevrilir. Bu çeviri **bir kez** yapılır; sonrasında program doğrudan donanım üzerinde çalışır.

Derleme Süreci Aşamaları

Lexical Analyzer (Sözcüksel Çözümleyici)

Kaynak program karakterlerini anlamlı lexical unit'lere (sözcüksel birimlere) dönüştürür: tanımlayıcılar (identifier), özel kelimeler, operatörler, noktalama. Yorumları (comment) görmezden gelir.

Örnek/Not: `int x = 5; → [int][x][=][5][;]`

Syntax Analyzer / Parser (Sözdizim Çözümleyici)

Lexical unit'lerden hiyerarşik **parse tree** (*ayrıştırma ağacı*) yapıları oluşturur. Bu ağaçlar programın sözdizimsel yapısını temsil eder.

Örnek/Not: Parse tree genellikle gerçek bir yapı değil, bilgi olarak oluşturulur.

Semantic Analyzer + Intermediate Code Generator (Semantik Çözümleyici + Ara Kod Üreticisi)

Semantik çözümleyici **type errors** (*tip hatası*) gibi sözdizimsel analizde yakalanamayan hataları tespit eder. Ara kod üreticisi, kaynak ile makine dili arasında bir **intermediate language** (*ara dil*) üretir.

Örnek/Not: Ara dil çoğunlukla assembly diline benzer, bazen ona eşdeğerdir.

Optimization (Eniyileme — Opsiyonel)

Programı daha küçük veya daha hızlı yapmak için ara kodu dönüştürür. Machine language optimizasyonu güç olduğundan çoğunlukla ara kod üzerinde yapılır. Daha fazla optimizasyon = daha uzun derleme süresi.

Örnek/Not: Üretim ortamı → yüksek optimizasyon; öğrenci lab → düşük optimizasyon

Code Generator (Kod Üreticisi)

Optimize edilmiş ara kodu eşdeğer makine dili programına çevirir.

Örnek/Not: Çıktı: donanım üzerinde doğrudan çalıştırılabilir makine kodu.

Symbol Table (Sembol Tablosu)

Kullanıcı tanımlı isimlerin tip ve özellik bilgilerini tutan derleme veritabanıdır. Sözcüksel ve sözdizim çözümleyiciler tarafından doldurulur; semantik çözümleyici ve kod üreticisi tarafından kullanılır.

Örnek/Not: Tüm derleme boyunca aktif bir veritabanı rolü üstlenir.

i Linking and Loading (Bağlama ve Yükleme)

Derlenmiş makine kodu doğrudan çalıştırılmaz; önce **linker** (*bağlayıcı*) tarafından sistem kütüphaneleriyle birleştirilmesi gerekir. Sonuç: **load module** (*yük modülü*) ya da **executable image** (*çalıştırılabilir görüntü*). Linker sistem programlarının giriş noktası adreslerini kullanıcı programındaki çağrılarının içine yerleştirir.

1.7.2 Pure Interpretation (Saf Yorumlama)

Saf yorumlamada program, **interpreter** (*yorumlayıcı*) adı verilen başka bir program tarafından **çeviri yapılmaksızın** doğrudan yürütülür. Yorumlayıcı, yüksek seviyeli dil ifadelerini işleyen bir **virtual machine** (*sanal makine*) simülasyonu olarak çalışır.

Tablo 1.8 — Pure Interpretation: Artı ve Eksiler

Avantajlar (+)	Dezavantajlar (-)
Kaynak düzeyinde hata mesajları (hangi satır, hangi değişken)	10–100 kat yavaş çalışma hızı
Kolay debug ve geliştirme	Her çalıştırmada komutlar yeniden çözümlenmeli
Platform bağımsız çalışma	Kaynak program + sembol tablosu bellekte → fazla bellek

1980'lere gelindiğinde saf yorumlama yüksek seviyeli dillerde nadiren kullanılır hale gelmişti. Ancak son yıllarda JavaScript ve PHP gibi web betik dilleriyle yeniden yaygınlaşmıştır. Tarihsel örnekler: APL, SNOBOL, Lisp.

1.7.3 Hybrid Implementation Systems (Hibrid Gerçekleme)

Derleyici ile saf yorumlayıcı arasında bir uzlaşma yöntemidir. Kaynak program önce **intermediate language** (*ara dile*) çevrilir, ardından bu ara kod yorumlanır. Kaynak çözümlemesi yalnızca bir kez yapıldığından saf yorumlamadan hızlıdır.

i Önemli Hibrid Örnekleri

Perl: Hataları önceden yakalamak ve yorumlayıcıyı basitleştirmek için kısmi derleme.

Java (ilk gerçeklemeler): Kaynak → **byte code** (*bayt kod*) → **Java Virtual Machine** (*Java Sanal Makinesi*) tarafından yorumlama. Byte code her platformda çalışır — taşınabilirlik garantisi.

JIT (Just-in-Time): Çalışma zamanında ara dil metodları ilk çağrıldığında makine koduna derlenir; derleme sonucu saklanır. Java ve .NET dilleri JIT kullanır.

1.7.4 Preprocessors (Ön-işleyiciler)

Bir **preprocessor** (*ön-işleyici*); derlemeden hemen önce programı işleyen **macro expander** (*makro genişletici*)dir. C'de iki yaygın kullanım:

C Ön-işleyici Örneği:

```
#include "myLib.h" // myLib.h içeriğini buraya kopyala

#define max(A, B) ((A) > (B) ? (A) : (B))

// Kullanım:
x = max(2 * y, z / 1.73);

// Ön-işleyici açar:
x = ((2 * y) > (z / 1.73) ? (2 * y) : (z / 1.73));
```

⚠ Side Effects Tehlikesi

Yukarıdaki max makrosunda parametrelerden biri yan etkiye sahipse (örn. `z++`) sorun çıkar: bir parametre iki kez değerlendirildiğinden `z` iki kez artar. Bu durum beklenmedik davranışlara yol açar.

Önemli Çıkarımlar (Key Takeaways)

- ✓ Üç temel gerçekleştirme yöntemi: Compilation · Pure Interpretation · Hybrid
- ✓ Compilation → en hızlı yürütme; bir kez çeviri, defalarca çalıştırma.
- ✓ Pure Interpretation → 10-100x yavaş; kaynak düzeyinde hata mesajları avantaj.
- ✓ Hybrid → orta hız; Java'nın byte code + JVM modeli.
- ✓ JIT: ilk çağrıda makine koduna derlenir; sonraki çağrılar hızlı çalışır.
- ✓ Von Neumann bottleneck: paralel bilgisayar araştırmalarının temel motivasyonu.
- ✓ Linker: kullanıcı programını sistem kütüphanelerine bağlar → executable image.

1.8 Programming Environments (Programlama Ortamları)

Bir **programming environment** (*programlama ortamı*), yazılım geliştirme sürecinde kullanılan araçlar bütünüdür. Basit bir metin editörü + derleyici ikilisinden, tümleşik büyük araç setlerine kadar geniş bir yelpazeyi kapsar. Dilin özellikleri tek başına değil, ortamın bütünüyle birlikte değerlendirilmelidir.

Ortam	Açıklama	Önemli Not
UNIX	1970'lerin ortasında dağıtıma girdi. Taşınabilir çoklu programlama işletim sistemi. Geniş araç yelpazesi (C, komut yorumlayıcısı vb.). GNOME, KDE gibi GUI katmanları artık Windows/Mac benzeri deneyim sunar.	<i>En eski; unix araçları uniform arayüzden yoksundu ama GUI'ler bunu giderdi.</i>
Borland JBuilder	Java geliştirme için tümleşik derleyici, editör, hata ayıklayıcı ve dosya sistemi. Grafik arayüzden erişilir.	<i>Karmaşık ve güçlü bir Java geliştirme platformu.</i>
Microsoft Visual Studio .NET	C#, VB.NET, JScript, F#, C++/CLI için kapsamlı araç koleksiyonu. Pencere tabanlı tek arayüzden erişilir.	<i>Sadece IDE değil, bir framework — uygulamanın ortak kodlarını da sağlar.</i>
NetBeans	Java'ya odaklı, aynı zamanda JavaScript, Ruby ve PHP desteği sunar.	<i>Hem geliştirme ortamı hem framework; Visual Studio ile bu özelliği paylaşır.</i>

Önemli Çıkarımlar (Key Takeaways)

- ✓ Programlama ortamı, dil kalitesini tamamlayan kritik bir bileşendir.
- ✓ Entegre araçlar (editör + derleyici + debugger) geliştirme ve bakım verimliliğini artırır.
- ✓ Modern IDE'ler yalnızca geliştirme ortamı değil, ortak kod sağlayan framework'tür.

QUIZ Çıkması Muhtemel Sınav Soruları ve Cevapları

Aşağıdaki sorular açık uçlu sınav formatındadır. Her soruyu önce kapatıp kendiniz yanıtlamaya çalışın, ardından cevabı okuyun.

S1. Orthogonality (ortogonallik) kavramını tanımlayın. Bir programlama dilinde orthogonality'nin hem az hem de fazla olmasının yarattığı sorunları açıklayın ve her durum için bir dil örneği verin.

► **CEVAP:**

Orthogonality: az sayıda primitive yapının az sayıda kuralla birleştirilerek dilin tüm kontrol ve veri yapılarını oluşturabilmesi; her kombinasyonun yasal ve anlamlı olmasıdır. Matematikteki dik vektörler gibi, her temel yapı diğerlerinden bağımsızdır.

AZ ORTHOGONALITY → Kural istisnaları artar, daha zor öğrenilir ve okunur.

Örnek — IBM: A komutu (Reg + Mem), AR komutu (Reg + Reg) gerektirir; 4 operand kombinasyonunun sadece 2'si yasal. Yüksek seviyeli örnek — C: array'lar fonksiyondan döndürülemez ama struct'lar döndürülebilir.

FAZLA ORTHOGONALITY → Kombinatoriyel patlama, anlaşılması güç karmaşıklık.

Örnek — ALGOL 68: her yapı bir türe sahip, kısıtlama yok; koşullu bir ifade atama'nın sol tarafında yer alabilir. Bu aşırı özgürlük gereksiz karmaşıklık yaratır.

S2. Von Neumann mimarisi ve imperative programlama dilleri arasındaki doğrudan ilişkiyi açıklayın. Fetch-execute cycle'ı adım adım anlatın.**► CEVAP:**

Von Neumann mimarisinde veri ve programlar aynı bellekte saklanır; CPU bellekten ayrıdır. Komutlar ve veriler piping ile CPU'ya, sonuçlar geri belleğe taşınır. Bu fiziksel kısıtlama imperative dillerin üç temel yapısını doğurur:

- Değişkenler (variables): bellek hücrelerini modeller
- Atama ifadeleri (assignment): piping operasyonunu temsil eder
- Döngüler (iteration): adjacent bellekte saklanan komutlar için en verimli tekrar

Fetch-Execute Cycle:

1. program_counter'i başlat
2. Sonsuz döngü:
 - a. PC'nin gösterdiği komutu bellekten al (fetch)
 - b. PC'yi bir sonraki komuta artır (increment)
 - c. Komutu çözümü – ne yapılacağını belirle (decode)
 - d. Komutu yürüt (execute)

S3. Derleme (compilation), saf yorumlama (pure interpretation) ve hibrid gerçekleştirme yöntemlerini karşılaştırın. Her birinin avantaj ve dezavantajlarını tartışın. JIT'i bu bağlamda konumlandırın.

► **CEVAP:**

COMPILATION: Kaynak kod bir kez makine diline çevrilir. En hızlı yürütme.

Avantaj: çeviri bir kez, çalışma binlerce kez hızlı. Dezavantaj: derleme ek adım.

Örnek: C, C++, COBOL.

PURE INTERPRETATION: Çeviri yok; yorumlayıcı kaynak kodu doğrudan yürütür.

Avantaj: kaynak düzeyinde hata mesajları, kolay debug.

Dezavantaj: 10-100 kat yavaş (her çalıştırmada komutlar yeniden çözülür).

Örnek: JavaScript, PHP.

HYBRID: Kaynak → ara dil (bir kez çözümleme) → yorumlama.

Avantaj: saf yorumlamadan hızlı, çapraz platform taşınabilirlik.

Dezavantaj: makine kodundan yavaş. Örnek: Java (byte code + JVM).

JIT (Just-in-Time): Hibridin özel bir türü. Çalışma zamanında ara dil metodları

ilk çağrıldığında makine koduna derlenir; derleme sonucu sonraki çağrılar için

saklanır. Böylece zamanla compilation hızına yaklaşır. Java ve .NET bu yöntemi kullanır.

S4. Type checking (tür denetimi) ve aliasing (takma adlama) kavramlarını güvenilirlik (reliability) bağlamında açıklayın. C ve Java'yı bu iki kriter açısından karşılaştırın.

► **CEVAP:**

TYPE CHECKING: Bir programdaki tip hatalarının derleme (compile-time) veya çalışma (run-time) zamanında tespit edilmesidir. Compile-time tercih edilir çünkü hatalar erken yakalanır ve düzeltme maliyeti düşer.

Java: neredeyse tüm tip hataları derleme zamanında yakalanır → yüksek güvenilirlik.

Orijinal C: subprogram parametre tipleri kontrol edilmiyordu; int yerine float beklenen fonksiyona gönderildiğinde hata verilmezdi → güvenilirlik sorunu.

ALIASING: Aynı bellek hücresine birden fazla isimle erişebilmektir. Tehlikelidir: birini değiştirmek diğerini de değiştirir; bu durum takip edilmesi güç hatalara yol açar.

C/C++: pointer aliasing mümkün — esneklik ↑ ama güvenilirlik ↓.

Java: pointer yok; referanslar için sınırlı aliasing mümkün.

Özet: Java tür denetimi + kısıtlı aliasing → yüksek güvenilirlik.

C minimum kısıtlama → yüksek esneklik, düşük güvenilirlik.

S5. Derleme sürecinin aşamalarını (lexical analyzer, syntax analyzer, intermediate code generator, semantic analyzer, optimization, code generator, symbol table) sırasıyla açıklayın. Her aşamanın giriş ve çıkışını belirtin.

► **CEVAP:**

1. LEXICAL ANALYZER (Sözcüksel Çözümleyici)

Giriş: kaynak program (karakter dizisi)

Çıkış: lexical units (tanımlayıcı, özel kelime, operatör, noktalama)

Yorumları yok sayar.

2. SYNTAX ANALYZER / PARSER (Sözdizim Çözümleyici)

Giriş: lexical units

Çıkış: parse trees (hiyerarşik sözdizim yapısı)

3. INTERMEDIATE CODE GENERATOR + SEMANTIC ANALYZER

Giriş: parse trees

Çıkış: intermediate code (ara dil kodu)

Semantik hatalar (tip hataları vb.) bu aşamada tespit edilir.

4. OPTIMIZATION (Opsiyonel)

Giriş: intermediate code

Çıkış: optimize edilmiş intermediate code

Daha küçük veya daha hızlı makine kodu için dönüşümler.

5. CODE GENERATOR

Giriş: optimize edilmiş intermediate code

Çıkış: machine language program (makine dili programı)

6. SYMBOL TABLE (tüm aşamalarda aktif)

İçerik: kullanıcı tanımlı isimlerin tip ve özellik bilgileri.

Sözcüksel + sözdizim çözümleyiciler doldurur;

semantik çözümleyici + kod üreticisi okur.

S6. Dil tasarımı trade-off (ödünleşim) kavramını açıklayın. Kitapta verilen üç somut çelişki örneğini tartışın ve her birinde hangi tasarımcı kararının alındığını belirtin.

► **CEVAP:**

Dil tasarımı değerlendirme kriterleri birbiriyle çelişebilir. Hoare (1973):

'Uzlaştırılması gereken pek çok önemli ama çelişen kriter vardır.'

ÖRNEKLERİ:

1. Reliability vs. Execution Cost:

Java: tüm dizi erişimlerinde index kontrolü yapılır.

→ Daha güvenilir ama daha yavaş.

C: index kontrolü yok → hızlı ama riskli.

Karar: Java güvenilirliği hıza tercih etti.

2. Writability vs. Readability (APL örneği):

APL çok güçlü dizi operatörlerine sahip; dev hesaplamalar çok az satırda yapılır.

→ Writability son derece yüksek.

Ama programlar o kadar yoğun ki McCracken 4 satırı anlamak için 4 saat harcadı.

Karar: APL yazılabilirliği okunabilirliğe feda etti.

3. Writability vs. Reliability (C++ vs Java):

C++ pointer manipülasyonu → yüksek esneklik (writability ↑).

Ama pointer güvenilirlik sorunları yarattığından Java'da pointer kaldırıldı.

Karar: Java güvenilirliği writability'ye tercih etti.

S7. Programlama dili kavramlarını öğrenmenin altı temel gerekçesini listeleyin ve her birini kısaca açıklayın.**► CEVAP:**

1. Fikir ifade kapasitesinin artması: kullanılan dilin ifade gücü düşünce derinliğini belirler; başka dillerin yapılarını bilmek bu kısıtlamaları aşmayı sağlar.
2. Uygun dil seçimi için gelişmiş altyapı: alışkanlıkla değil bilinçli değerlendirmeye projeye en uygun dil seçilebilir.
3. Yeni dil öğrenme kapasitesinin artması: temel kavramlar edinilince yeni dillerin yapıları daha hızlı kavranır.
4. Uygulama öneminin daha iyi anlaşılması: dilin neden belirli biçimde tasarlandığını kavramak, onu daha etkin ve performanslı kullanmayı sağlar.
5. Bilinen dillerin daha verimli kullanımı: büyük dillerde pek çok özellik bilinmez; kavramsal eğitim bu boşlukları kapatır.
6. Bilişimin genel ilerlemesine katkı: bilinçli dil seçimi yapan topluluk, uzun vadede daha iyi dillerin yaygınlaşmasına katkı sağlar.

REVIEW Kitap Bölüm Sonu Soruları ve Tam Çözümleri

Kitabın 'Review Questions' bölümünden seçilmiş sorular ve kapsamlı yanıtları. Bu sorular sınav formatıyla örtüşmektedir.

S1. Why is it useful for a programmer to have some background in language design, even though he or she may never actually design a programming language? (Bir programcının dil tasarımı konusunda arka plan bilgisine sahip olması neden faydalıdır?)

► ÇÖZÜM:

Dil tasarımı bilgisi dört somut kazanım sağlar:

- (1) Düşünce ve ifade genişliği: Farklı dillerin yapılarını bilmek, kendi kullandığı dilde bile bu yapıları simüle etme imkânı tanır.
- (2) Bilinçli dil seçimi: Hangi dili seçeceğini bilen programcı, projeye en uygun aracı alışkanlıkla değil, gerekçeli tercihle belirler.
- (3) Kolay dil öğrenimi: Kavramsal altyapı, yeni dillerin öğrenimini hızlandırır.
- (4) Uygulama farkındalığı: Dilin neden belirli biçimde tasarlandığını anlamak, performans odaklı ve hatalardan uzak kod yazımına katkı sağlar.

**S8. How can user-defined operator overloading harm the readability of a program?
(Kullanıcı tanımlı operatör aşırı yükleme okunabilirliği nasıl zedeler?)****► ÇÖZÜM:**

Operatör aşırı yükleme (operator overloading) kendi başına zararlı değildir.

Örneğin + operatörünü hem tamsayı hem ondalıklı toplama için kullanmak mantıklıdır ve dili basitleştirir.

Sorun, kullanıcı tanımlı ve anlamsız aşırı yüklemelerde ortaya çıkar.

Örnek:

+ operatörü iki tek boyutlu dizi arasında 'her iki dizinin tüm elemanlarının toplamı' anlamında tanımlanırsa, vektör toplamı beklentisiyle okuyan bir kişi için program tamamen yanıltıcı hale gelir.

Daha aşırı: + iki vektör arasında 'ilk elemanlarının farkı' anlamında tanımlanırsa, hem yazar hem okuyucu için programı anlamak neredeyse imkânsız olur.

Özetle: kullanıcı tanımlı aşırı yüklemeler dile ek anlam katmanları ekler; okuyucu bu katmanları bilmeden kodu doğru yorumlayamaz.

S9. What is one example of a lack of orthogonality in the design of C? (C'nin tasarımındaki orthogonality eksikliğine bir örnek verin.)

► **ÇÖZÜM:**

C'de pek çok orthogonality ihlali bulunmaktadır:

En belirgin örnek — Array vs Struct asimetrisi:

- Struct (kayıt) türleri fonksiyondan döndürülebilir.
- Array (dizi) türleri fonksiyondan döndürülemez.

→ Aynı kural iki yapısal veri tipine farklı uygulanıyor.

Ek örnekler:

- Parametreler değer ile geçilir; ancak array'lar referans ile geçilir (array adı, ilk elemanın adresine dönüşür).
- static anahtar kelimesi bağlama göre farklı anlam taşır:
 - Fonksiyon içindeki değişken: compile-time'da oluşturulur.
 - Fonksiyon dışındaki değişken: dosya kapsamıyla sınırlıdır.

S19. What is the name of the category of programming languages whose structure is dictated by the von Neumann computer architecture? (Von Neumann mimarisince yapısı belirlenen dil kategorisi nedir?)

► **ÇÖZÜM:**

Bu diller IMPERATIVE LANGUAGES (Buyurumsal Diller) olarak adlandırılır.

Von Neumann mimarisinden miras aldıkları üç temel yapı:

1. Variables (Değişkenler) — bellek hücrelerini modeller
2. Assignment statements — piping operasyonunu temsil eder
3. Iterative repetition — adjacent bellekte en verimli tekrar biçimi

Örnekler: C, C++, Java, C#, Python, COBOL, Fortran, Ada

S24. What are the three general methods of implementing a programming language? (Bir programlama dilini gerçeklemenin üç genel yöntemi nelerdir?)**► ÇÖZÜM:****1. COMPILATION (Derleme):**

Kaynak kod makine diline çevrilir. Çeviri bir kez yapılır; program doğrudan donanım üzerinde çalışır. En hızlı yürütme.

Örnekler: C, C++, COBOL.

2. PURE INTERPRETATION (Saf Yorumlama):

Çeviri yapılmaz; yorumlayıcı kaynak kodu doğrudan yürütür.

Avantaj: kaynak düzeyinde hata mesajları.

Dezavantaj: 10-100 kat yavaş; her çalıştırmada komutlar yeniden çözümlenir.

Örnekler: JavaScript, PHP (geleneksel).

3. HYBRID IMPLEMENTATION (Hibrid Gerçekleme):

Kaynak önce ara dile çevrilir (bir kez), ardından ara kod yorumlanır.

Saf yorumlamadan hızlı; makine kodundan yavaş.

Örnekler: Java (byte code + JVM), Perl, .NET (JIT).

S26. What role does the symbol table play in a compiler? (Derleyicide sembol tablosu hangi rolü üstlenir?)**► ÇÖZÜM:**

Sembol tablosu (symbol table), tüm derleme süreci boyunca aktif olan bir veritabanıdır.

İÇERİK: Kullanıcı tanımlı her ismin (değişken, fonksiyon, tip vb.)

- tip bilgisi (int, float, struct vb.)
- özellik bilgileri (kapsam, boyut, adres vb.)

KİM DOLDURUR: Lexical analyzer ve syntax analyzer

KİM KULLANIR: Semantic analyzer ve code generator

NEDEN KRİTİK:

- Semantic analyzer tip hatalarını sembol tablosundaki bilgiye dayanarak tespit eder.
- Code generator, makine kodu üretirken tip ve özellik bilgilerine ihtiyaç duyar.

S27. What does a linker do? (Bağlayıcı (linker) ne yapar?)**► ÇÖZÜM:**

Linker (bağlayıcı); derlenmiş kullanıcı programını sistem kütüphaneleri ve diğer önceden derlenmiş programlarla birleştirir.

ADIMLAR:

1. Kullanıcı programındaki sistem program çağrılarını tespit eder.
2. İlgili sistem programlarını kütüphanelerde bulur.
3. Sistem programlarının giriş noktası adreslerini (entry point) kullanıcı programındaki çağrı noktalarına yerleştirir.

ÇIKTI: Load module veya executable image — çalıştırılabilir program.

Linker olmadan derlenmiş bir program input/output gibi temel işlemleri bile gerçekleştiremez.

S28. Why is the von Neumann bottleneck important? (Von Neumann darboğazı neden önemlidir?)**► ÇÖZÜM:**

Von Neumann bottleneck; bellek ile CPU arasındaki bağlantı hızının tüm sistem performansını sınırlamasıdır.

NEDEN OLUŞUR:

Komutlar CPU'ya taşınmadan yürütülemez. CPU genellikle komutları belleğe taşıyabileceğinden daha hızlı işleyebilir. Bu iletim hızı sistemin gerçek hız tavanını belirler.

NEDEN ÖNEMLİ:

1. Von Neumann mimarisi üzerine tasarlanan tüm sistemleri etkiler.
2. Fonksiyonel dillerin yaygınlaşamamasının başlıca nedenidir: von Neumann dışı bir donanım olmadan fonksiyonel programlar imperative programlar kadar verimli çalışamaz.
3. Paralel bilgisayar araştırmalarının temel motivasyonu olmuştur.